

Homework 5 Lab Assignment (50 points)

A Simple Guitar String Simulation

Marina Bosi – Music 320 Intro to Audio Signal Processing

11/7/25

Introduction

In this problem, we are going to analyze and implement a very simple simulation of a plucked guitar string. We will model a plucked guitar string¹ as the [damped feedback comb filter](#) shown in the figure below.² This simple LTI system corresponds to the following difference equation:

$$y[n] = x[n] + ay[n - M]$$

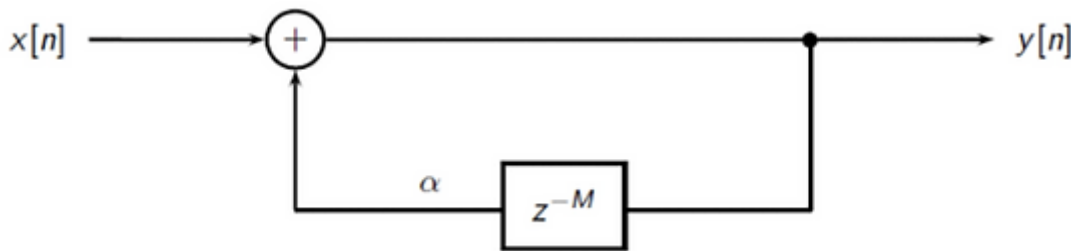


Figure 1: Damped Feedback Comb Filter

In this model, the delay time M represents the time for traveling waves on the string to travel from the bridge to the fret and back and the factor a represents the (frequency-independent) damping over that traversal. Knowing that the string length determines the pitch of a plucked string, we can assume that the delay time M will control the pitch of our simulated string with shorter M giving higher pitches and longer M giving lower pitches.

¹JOS, “Physical Audio Signal Processing”, Ch 6.

²Replacing the decay factor a with an order-two low pass averaging filter $\frac{1}{2}[1, 1]$ gives the famous “[Karplus and Strong](#)” algorithm which you can hear [here](#).

To simulate a plucked guitar string with this model, we'll start with a random input $x[n]$ which is M samples long and feed it through this filter. After the first M samples pass through the filter, this system will keep repeating the M -block decayed by an additional factor of a for each time the block has passed through the filter. (To work with a decay time that is independent of M , we'll work in terms of κ defined by $a \equiv \kappa^M$.)

We'll implement, analyze, and play some music with this simplified guitar string simulation.

Part I: Analysis

- 1) Please write the transfer function $H(z) = Y(z)/X(z)$ for this discrete time filter.
- 2) Please find the poles of $H(z)$.
 - a) Write the poles in polar coordinates $p_i = r_i e^{j\theta_i}$. What is the smallest (non-zero) angle θ_i as a function of M ?
 - b) Please plot the poles in the complex plane for $M = 10$ and $a = 0.9^{10}$. (Please show the unit circle on your plot.)
- 3) Please compute and plot the amplitude response of this discrete-time filter for $M = 10$ and $a = 0.9^{10}$. Please describe how the r_i and θ_i affect the amplitude response? Looking at the amplitude response, what is the (positive) frequency location of the first peak?
- 4) What is the normalized radian frequency of a sinusoid whose period equals M ? What is this frequency in Hz when the sampling frequency is f_s ? How does this sinusoid frequency relate to the frequency of the first peak of the amplitude response of this filter?
- 5) Working with $f_s = 8$ kHz, select M so that the filter's first peak frequency is closest to the A_4 440 Hz frequency. Moving up 2 semi-tones to B_4 , which value for M gets us closest to its $(440 \times 2^{\frac{2}{12}}) = 493.88$ Hz frequency.
- 6) If we are working with $f_s = 8$ kHz, at which value for κ do we have an amplitude decay by a factor of 10 over one second? Note that by setting the metronome at 60 beats per minute (60 bpm), with a 4/4 time signature each quarter note lasts about one second.

Part II: Simulation

- 7) Please write a python function `Guitar(N, M, kappa)` which uses `np.random.randn()` to feed a random signal M samples long as input to the damped feedback filter with delay M and decay $a = \kappa^M$ and which returns an output signal N samples long.
- 8) Please plot the output of this function for $N = 16,000$ using the M for A_4 from question I.5 and the κ from question I.6 above.

Part III: Making Music

- 9) Please write a python function `freq(note)` that takes a note (as a text character) in octave 4 and returns its frequency in Hz. In doing so, you may find this python dictionary helpful: `semitones = {'C': -9, 'D': -7, 'E': -5, 'F': -4, 'G': -2, 'A': 0, 'B': 2}` where the frequency of each note is $440 \text{ Hz} \cdot 2^{\frac{\text{semitones}}{12}}$ (octave 4).

[Note: This only considers the 7 natural notes in the octave.]

- 10) Please write a function `PlayNotes(notes, fs, octave_shift = 0)` that takes as input a python list of notes and a sampling rate and then creates an output signal `fs * (len(notes) + 1)` long which is initialized to all zeros and which adds in each note `fs` samples later computed over `2*fs` samples of length. Have the function return an audio player `IPython.display.Audio(output, rate = fs)` so you can hear the output signal. When `octave_shift` is non-zero, multiply all frequencies by $2^{\text{octave_shift}}$ so you can shift the notes up and down to other octaves. Also, if the note is “P” consider it a pause and shift ahead one time step (f_s samples) to the next note.
- 11) Please test your function at $f_s = 8 \text{ kHz}$ with the following note sequence shifted down by one octave (octave 3):

```
[ 'C', 'C', 'G', 'G', 'A', 'A', 'G', 'P'
, 'F', 'F', 'E', 'E', 'D', 'D', 'C', 'P'
, 'G', 'G', 'F', 'F', 'E', 'E', 'D', 'P'
, 'G', 'G', 'F', 'F', 'E', 'E', 'D', 'P'
, 'C', 'C', 'G', 'G', 'A', 'A', 'G', 'P'
, 'F', 'F', 'E', 'E', 'D', 'D', 'C']
```

Additional Suggestions (Just for fun – no extra credit):

- Add in flats and naturals to get the full 12 note octave so you can describe notes such as “C#” (a.k.a “D $\flat$ ”).
- Add in the octave of notes so that you can go beyond notes from just one octave. (So now notes can be written as “A4” and “C3#”.)
- Add in information about note length (right now all notes are quarter notes) and take that into account both in how much time is allocated to each note and to the κ being used for the decay of each note. (Use $\sqrt{\kappa}$ for half notes, κ^2 for eighth notes, etc.)
- Allow the playing speed (in bpm) to change from the default 60 bpm. You can implement this by letting the basic quarter note length go from f_s to $\frac{60}{BPM} \cdot f_s$.